INSTITUTO FEDERAL DE MATO GROSSO DO SUL
CAMPUS NOVA ANDRADINA

NOTAS DE AULA DE

LINGUAGEM E TÉCNICAS DE PROGRAMAÇÃO

Prof. Me. Luiz F. Picolo

NOVA ANDRADINA - MS

Atualizado em 14 de abril de 2021

2 Orientação a Objetos com JavaScript

Dedique-se aos estudos para que eles o façam melhor para a sociedade e para si mesmo.
Alvaro Granha Loregian

Esse capítulo tem como objetivo aplicar o conhecimento adquirido anteriormente no Capítulo **??** utilizando a linguagem de programação JavaScript.

JavaScript, segundo Trasviña 2021, tem fortes capacidades de programação orientada a objetos, apesar de ocorrerem algumas discussões devido às diferenças da orientação a objetos no JavaScript em comparação com outras linguagens. Assim, no Capítulo **??** foi realizada uma introdução à programação orientada a objetos, e neste, será demonstrado os conceitos de programação orientada a objetos no JavaScript.

2.1 Classes

Classes em JavaScript[1] são introduzidas no ECMAScript[2] 2015 e são simplificações da linguagem para as heranças baseadas nos protótipos. A sintaxe para classes não introduz um novo modelo de herança de orientação a objetos em JavaScript. Classes em JavaScript provêm uma maneira mais simples e clara de criar objetos e lidar com herança.

```
let Usuario = {
  nome: 'Luiz Picolo',
  profissao: 'Professor'
}

console.log(Usuario.nome)
```

```
class Usuario {
  constructor(){
    this.nome = 'Luiz Picolo';
    this.profissao =
    'Professor'
  }
}

usuario = new Usuario();
console.log(usuario.nome)
```

Figura 1 – Sintaxes de classes em JavaScript

Fonte: O Autor

Para definir uma classe é usando uma declaração de classe. Para declarar uma classe, deve-se usar a palavra reservada **class** seguida pelo nome da classe desejada.

[1] Para mais detalhes acesse: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Classes>

[2] Para mais detalhes acesse: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>

```
class Retangulo {

}
```

Outras forma de escrita de uma classe, também chamadas Expressões de Classes (*class expression*), podem ser usadas para fazer sua definição. Contudo, utilizaremos apenas a notação mais nova e comumente encontrada em outras linguagens, ou seja, a notação *class*.

```
// sem nome
let Retangulo = class {

};

// nomeada
let Retangulo = class Retangulo {

};
```

E para que possamos criar uma instância (objeto) da classe, utilizamos o seguinte trecho de código abaixo.

```
let retangulo = new Retangulo();
```

2.2 Construtor e Atributos

Como dito no Capítulo **??** as características que descrevem um objeto são chamadas na orientação a objeto de atributos. Assim, toda classe pode conter atributos que lhe dão as características necessárias para a criação do objeto.

Para que possamos criar nossos atributos podemos fazer de duas formas, uma usando funções diretas ou por meio do Construtor. Um contrutor é um tipo especial de "método" para criar e iniciar um objeto criado pela classe. Só pode existir um método construtor dentro da classe. Um erro de sintáxe **SyntaxError** será lançado se a classe possui mais do que uma ocorrência do método construtor.

Para iniciarmos os atributos com o construtor usamos a palavra reservada **constructor** seguida dos valores dos atributos desejados.

```
class Retangulo {
  constructor(altura, largura) {
    this._altura = altura;
    this._largura = largura;
  }
}
```

2.3 Métodos

Métodos são ações que o objeto pode realizar e são responsáveis pela troca de mensagem entre os objetos **emissores** e o **receptores**. As técnicas de programação orientada a objetos recomendam que a estrutura de um objeto e a implementação de seus métodos devem ser tão privativos como possível. Normalmente, os atributos de um objeto não devem ser visíveis externamente. Da mesma forma, de um método deve ser suficiente conhecer apenas sua especificação, sem necessidade de saber detalhes de como a funcionalidade que ele executa é implementada[3].

Figura 2 – Troca de mensagens entre objetos.

Fonte: O Autor

Assim, para definir um método, devemos atribuir uma função a uma classe que, depois disso, poderá ser chamado como método do objeto usando o mesmo nome ao qual foi atribuída a função.

```
class Retangulo {
  constructor(altura, largura) {
    this._altura = altura;
    this._largura = largura;
```

[3] Veja mais em: <https://www.devmedia.com.br/orientacao-a-objetos-parte-ii/7161>

```
5     }
6
7     calcularArea() {
8       return this._altura * this._largura;
9     }
10  }
```

Para invocarmos o método, basta instânciar-mos a classe e criar assim um novo objeto.

```
1   const objeto = new Retangulo(10, 10);
2   console.log(objeto.calculaArea());
```

Contudo, existe aqui um erro semântico. Quando queremos saber a área de uma figura geométrica, deveríamos "perguntar" a está figura sua área e não solicitar o calculo. Logo, podemos resolver isso, criando um novo método que retorna apenas a área e que utiliza o método **calcularArea**.

```
1   class Retangulo {
2     constructor(altura, largura) {
3       this._altura = altura;
4       this._largura = largura;
5     }
6
7     area(){
8       return this.calcularArea();
9     }
10
11    calcularArea() {
12      return this._altura * this._largura;
13    }
14  }
15
16  const objeto = new Retangulo(10, 10);
17  console.log(objeto.area());
```

2.3.1 Método Get

Os métodos criados acima funcionam corretamente. Porém, podemos melhorar um pouco mais a semântica do objeto. Para isso usaremos a sintáxe **Get**.

Às vezes é desejável que se permita acesso a uma propriedade que retorna um valor computado dinamicamente, ou você pode querer refletir o status de uma variável interna sem requerer o uso de chamadas de método explícitas. Em Javascript, isso pode ser feito com o uso de um **getter**. Não é possível simultaneamente ter um **getter** associado a uma propriedade e a mesma possuir um valor, embora seja possível usar um **getter** e um setter em conjunto para criar algo como uma pseudo-propriedade.

```
1  class Retangulo {
2    constructor(altura, largura) {
3      this._altura = altura;
4      this._largura = largura;
5    }
6
7    get area(){
8      return this.calcularArea();
9    }
10
11   calcularArea() {
12     return this._altura * this._largura;
13   }
14 }
15
16 const objeto = new Retangulo(10, 10);
17 console.log(objeto.area);
```

2.3.2 Método Set

Já o **setter** pode ser usado para executar uma função sempre que se tenta mudar uma propriedade específica. **Setters** são geralmente usados em conjunto com **getters**, para criar um tipo de pseudo-propriedade. No entanto é impossível ter-se um **setter** para uma propriedade que contenha um valor real.

```
1  class Retangulo {
2
3    set altura(altura){
```

```
4         this._altura = altura;
5       }
6
7       set largura(largura){
8         this._largura = largura;
9       }
10
11      get area(){
12        return this.calcularArea();
13      }
14
15      calcularArea() {
16        return this._altura * this._largura;
17      }
18    }
19
20    const objeto = new Retangulo();
21    objeto.largura = 10;
22    objeto.altura = 10;
23    console.log(objeto.area);
24
25    objeto.largura = 100;
26    objeto.altura = 100;
27    console.log(objeto.area);
```

O setter se torna muito útil nesse caso pois não precisaremos criar uma novo objeto, apenas alteramos seus atributos (seu estado) para obter novos valores.

2.3.3 Método Static

A palavra chave **static** define um método estático para a classe. Métodos estáticos não são chamados na instâncias da classe. Em vez disso, eles são chamados na própria classe. Geralmente, são funções utilitárias, como funções para criar ou clonar objetos. Chamadas a métodos estáticos são feitas diretamente na classe e não podem ser feitas em uma instância da classe. Métodos estáticos são comumente utilizados como funções utilitárias[4].

```
1    class Retangulo {
2
```

[4] Veja mais em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Classes/static>

```
3     // Codigo anterior
4
5     // Metodo adicionado
6     static formaGeometrica(objeto){
7       if (objeto._altura == objeto._largura){
8         return ‘‘A forma geometrica é um Quadrado’’
9       } else {
10        return ‘‘A forma geometrica é um Retangulo’’
11      }
12    }
13  }
14
15  const objeto = new Retangulo();
16  objeto.largura = 10;
17  objeto.altura = 10;
18  console.log(Retangulo.formaGeometrica(objeto));
19  console.log(objeto.area);
```

2.4 Exercícios de fixação

1. Uma empresa de notícias **IFNews** necessita desenvolver um portal de notícias. Após criada toda a parte documental, você, que é desenvolvedor/desenvolvedora na empresa, deverá iniciar a criação dos primeiros códigos. Sua primeira tarefa é a criação da classe Noticia. A classe deverá ser codificada seguindo, primordialmente, as definições abaixo:

   a) **Nome da classe**: Noticia

   b) **Atributos**: Título, Data da Publicação, Resumo e Texto (Outros atributos podem ser adicionados)

   c) **Métodos:** Mostrar Noticia (Outros métodos podem ser adicionados)

   Ao instanciar a classe e invocar o método **mostrar notícia** o resultado final deverá ser semelhante ao descrito abaixo (Lembre-se que o conteúdo no título, a data e o texto dependerá do **estado** adicionado a seu objeto):

   Resultado esperado:

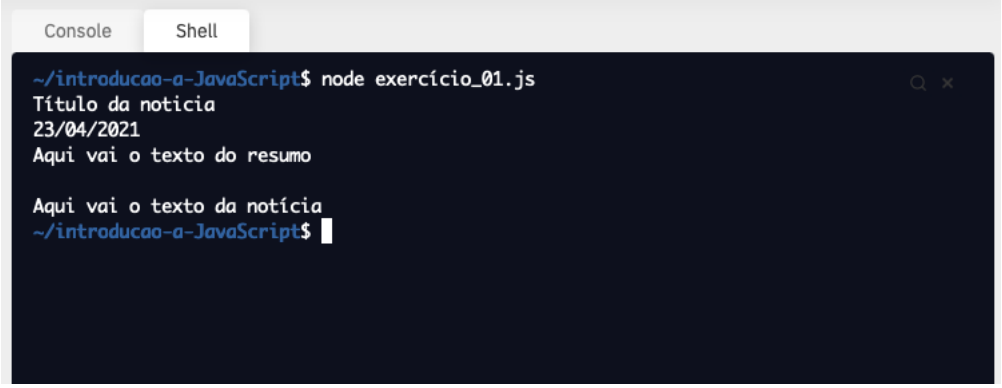

Figura 3 – Resultado esperado

Fonte: O Autor

2.5 Herança

A definição de herança, já abordada no Capítulo anterior, nós diz que: a herança é uma maneira de reutilizar código a medida que podemos aproveitar os atributos e métodos de classes já existentes para gerar novas classes mais específicas que aproveitarão os recursos da classe hierarquicamente superior [Ruiz 2008].

Para realizar a reutilização ja citada, no ES6 foi adicionada a palavra reservada ***extends***. Uma classe pode herdar atributos e métodos de outra classe por meio da extensão da classe desejada.

```
class Retangulo {

  // Codigo anterior
  ...
}

class Quadrado extends Retangulo{
  constructor(lado) {
    super(lado, lado);
  }
}

let objeto = new Quadrado(100)
console.log(objeto.area)
```

Observem a palavra **super**. A palavra reservada **super** é usada para acessar o objeto pai de um objeto, em outros casos, é usada para acessar a classe pai de uma classe.

Neste caso, precisamos observar alguns pontos:

- No construtor, o **super** deve ser chamado antes de qualquer acesso à **this**;
- Caso a classe que está realizando *extends* não defina o constructor ele será definido e chamado implicitamente passando todos os parâmetros.

```
// Errado
class Quadrado extends Retangulo{
  constructor(lado) {
    this._lado = lado;
    super(lado, lado);
  }
}

// Correto
class Quadrado extends Retangulo{
  constructor(lado) {
    super(lado, lado);
    this._lado = lado;
  }
}
```

2.6 Super classe e Subclasse

Atrelado com o conceito de herança possuímos as definições de **Super classe** e **Subclasses**. O uso da palavra **extends** indica que a classe deveria ser baseado em um objeto padrão. Isso é chamado de Super Classe (***superclass***) e a classe derivada é a Subclasse (***subclass***) [Haverbeke 2018]

```
class Retangulo {
   ...
}

class Quadrado extends Retangulo{
   ...
}
```

Ocasionalmente, é útil saber se um objeto foi derivado de uma classe. Para isso, o JavaScript fornece um operador binário chamado **instanceof** [Haverbeke 2018].

```
console.log(new Quadrado(2) instanceof Retangulo) \\ True
```

Mas e **Retangulo** é instância de alguma classe? Neste caso sim, todos os objetos em javaScript herdam a classe pai nomeado de **Object**

```
  console.log(new Quadrado(2) instanceof Object) \\ True
  console.log(new Retangulo(2, 2) instanceof Object) \\ True
```

2.7 Exercícios de fixação

1. A empresa de notícias **IFNews** contratou você para desenvolver o portal de notícias, o qual teve seu início no exercício anterior. Sua primeira tarefa foi a criação da classe Noticia que possuía as seguintes definições:

   a) **Nome da classe**: Noticia

   b) **Atributos**: Título, Data da Publicação, Resumo e Texto (Outros atributos podem ser adicionados)

   c) **Métodos:** Mostrar Noticia (Outros métodos podem ser adicionados)

Após desenvolver a tarefa, a empresa criou uma nova solicitação. O portal deve possuir **notícias em destaque**, as quais devem conter uma imagem seguida de um título e um resumo. Como você já criou uma notícia que contem parte do que o cliente necessita, a única tarefa será criar uma **subclasse** que herdará as características e ações da **Super Classe** e adicionará a característica desejada.

1. **Nome da classe**: NoticiaDestaque
2. **Atributos**: ImagemDestaque (apenas o caminho da imagem)
3. **Métodos:** Mostrar Destaque (Outros métodos podem ser adicionados)

Resultado esperado:

```
~/introducao-a-JavaScript$ node exercício_01.js
http://www.ifms.edu.br/imagem.jpg
Título da noticia
23/04/2021
Aqui vai o texto do resumo

Aqui vai o texto da notícia
~/introducao-a-JavaScript$
```

Figura 4 – Resultado esperado

Fonte: O Autor

Referências

HAVERBEKE, M. *Eloquent JavaScript: a modern introduction to programming*. [S.l.]: No Starch Press, 2018. Citado 2 vezes nas páginas 10 e 11.

RUIZ, E. E. S. *IBm1030 Programação Orientada a Objetos*. 2008. Disponível em: <http://dcm.ffclrp.usp.br/~evandro/ibm1030/constru/heranca.html>. Citado na página 9.

TRASVIñA, F. *Introdução ao JavaScript Orientado a Objeto*. 2021. Disponível em: <https://developer.mozilla.org/pt-BR/docs/conflicting/Learn/JavaScript/Objects>. Nenhuma citação no texto.